Command injection in Java

From OWASP

- 1 Status
- 2 Overview
- 3 Examples
  - 3.1 Example 1
  - 3.2 Example 2
- 4 Best Practices

Status

Review

Overview

Command injection vulnerabilities allow an attacker to inject arbitrary system commands into an application. The commands execute at the same privilege level as the Java application and provides an attacker with functionality similar to a system shell. In Java, Runtime.exec is often used to invoke a new process, but it does not invoke a new command shell, which means that chaining or piping multiple commands together does not usually work. Command injection is still possible if the process spawned with Runtime.exec is a command shell like command.com, cmd.exe, or /bin/sh.

Examples

Example 1

The code below allows a user to control the arguments to the Window's *find* command. While the user does have full control over the arguments, it is not possible to inject additional commands. For example, inputting "test & del file" will not cause the *del* command to execute, since Runtime.exec tokenizes the command string and then invokes the *find* command using the parameters "test", "&", "del", and "file."

```
import java.io.*;

public class Example1 {
        public static void main(String[] args)
        throws IOException {
                if(args.length != 1) {
                        System.out.println("No arguments");
                        System.exit(1);
                }
                Runtime runtime = Runtime.getRuntime();
                Process proc = runtime.exec("find" + " " + args[0]);

                InputStream is = proc.getInputStream();
                InputStreamReader isr = new InputStreamReader(is);
                BufferedReader br = new BufferedReader(isr);

                String line;
                while ((line = br.readLine()) != null) {
                        System.out.println(line);
                }
```

```
        }
}
```

Example 2

The code below invokes the system shell in order to execute a non-executable command using user input as parameters. Non-executable Window's commands such as *dir* and *copy* are part of the command interpreter and therefore cannot be directly invoked by Runtime.exec. In this case, command injection is possible and an attacker could chain multiple commands together. For example, inputting ". & echo hello" will cause the *dir* command to list the contents of the current directory and the *echo* command to print a friendly message.

```
import java.io.*;

public class Example2 {
        public static void main(String[] args)
        throws IOException {
                if(args.length != 1) {
                        System.out.println("No arguments");
                        System.exit(1);
                }
                Runtime runtime = Runtime.getRuntime();
                String[] cmd = new String[3];
                cmd[0] = "cmd.exe" ;
                cmd[1] = "/C";
                cmd[2] = "dir " + args[0];
                Process proc = runtime.exec(cmd);

                InputStream is = proc.getInputStream();
                InputStreamReader isr = new InputStreamReader(is);
                BufferedReader br = new BufferedReader(isr);

                String line;
                while ((line = br.readLine()) != null) {
                        System.out.println(line);
                }
        }
}
```

Best Practices

Developers should avoid invoking the shell using Runtime.exec in order to call operating system specific commands and should use Java APIs instead. For example, instead of calling *ls* or *dir* from the shell use the Java File class and the list function. If it is necessary to accept user input and pass it to Runtime.exec, then use regular expressions to validate the input.

1. **Traduccion en Español:** Inyección De Comandos En Java

Retrieved from "https://www.owasp.org/index.php?title=Command_injection_in_Java&oldid=206862"

Categories: FIXME/pls review | Java

- This page was last modified on 20 January 2016, at 07:01.
- Content is available under Creative Commons Attribution-ShareAlike unless otherwise noted.

-
- Open Web Application Security Project, OWASP, Global AppSec, AppSec Days, AppSec California, SnowFROC, LASCON, and the OWASP logo are trademarks of the OWASP Foundation.